

Waffle Charts, Word Clouds, and Regression Plots

What I'm Doing

In this lab, I explore three distinct but complementary visualization techniques using Python: **waffle charts** for proportional categorical display, **word clouds** for text-frequency analysis, and **regression plots** using Seaborn for statistical trend visualization. All work is centered on the Canadian immigration dataset (1980–2013).

1. Setting Up

I start by installing and importing the required libraries. I need matplotlib and pandas for core data handling, seaborn for statistical plotting, and the wordcloud package for generating word clouds.

```
!pip install matplotlib pandas seaborn wordcloud pywaffle
%matplotlib inline
```

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
import pandas as pd
from PIL import Image
import seaborn as sns
import wordcloud
from wordcloud import WordCloud, STOPWORDS
import urllib
from pywaffle import Waffle
```

```
mpl.style.use('ggplot')
```

```
print('Matplotlib version:', mpl.__version__)
print('Seaborn version:', sns.__version__)
print('WordCloud version:', wordcloud.__version__)
```

```
Matplotlib version: 3.10.9
Seaborn version: 0.13.2
WordCloud version: 1.9.6
```

2. Getting to Know the Dataset

I load the Canadian immigration dataset into a pandas DataFrame.

```
df_can = pd.read_csv(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/'
```

```
'IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/Canada.csv')
```

Let's inspect the first five rows to understand the structure.

```
df_can.head()
```

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...	2005	:
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	3436	
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1223	
2	Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	...	3626	
3	American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	...	0	
4	Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	...	0	

5 rows × 39 columns

The dataset contains annual immigration counts per country from 1980 to 2013, along with metadata like Continent and Region.

```
print(df_can.shape)
```

```
(195, 39)
```

That's 195 countries and 39 columns. I set the `Country` column as the index for easier row-based access.

```
df_can.set_index('Country', inplace=True)
```

This makes `df_can` indexed by country name, so I can use `.loc[]` to look up specific nations directly.

3. Waffle Charts

A **waffle chart** displays proportions as **square tiles** in a grid (**normally created to display progress toward goals**) - each tile represents a fraction of the whole. I'll build one from scratch for Denmark, Norway, and Sweden, then wrap it into a reusable function.

First, I subset the data for these three Scandinavian countries.

```
df_dsn = df_can.loc[['Denmark', 'Norway', 'Sweden'], :]  
df_dsn
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005
Country												
Denmark	Europe	Northern Europe	Developed regions	272	293	299	106	93	73	93	...	62
Norway	Europe	Northern Europe	Developed regions	116	77	106	51	31	54	56	...	57
Sweden	Europe	Northern Europe	Developed regions	281	308	222	176	128	158	187	...	205

3 rows × 38 columns

Building a Waffle Chart from Scratch (7 Steps)

Step 1 - Compute proportions of each category.

```
total_values = df_dsn['Total'].sum()
category_proportions = df_dsn['Total'] / total_values
pd.DataFrame({"Category Proportion": category_proportions})
```

Category Proportion	
Country	
Denmark	0.322557
Norway	0.192409
Sweden	0.485034

Step 2 - Define the chart dimensions.

```
width = 40
height = 10
total_num_tiles = width * height
print(f'Total number of tiles is {total_num_tiles}.')
```

Total number of tiles is 400.

Step 3 - Compute how many tiles each category gets.

```
tiles_per_category = (category_proportions * total_num_tiles).round().astype(int)
pd.DataFrame({"Number of tiles": tiles_per_category})
```

Number of tiles	
Country	
Denmark	129
Norway	77
Sweden	194

Step 4 - Create and populate a matrix representing the waffle grid.

```
# initialize the waffle chart as an empty matrix
waffle_chart = np.zeros((height, width), dtype=np.uint)

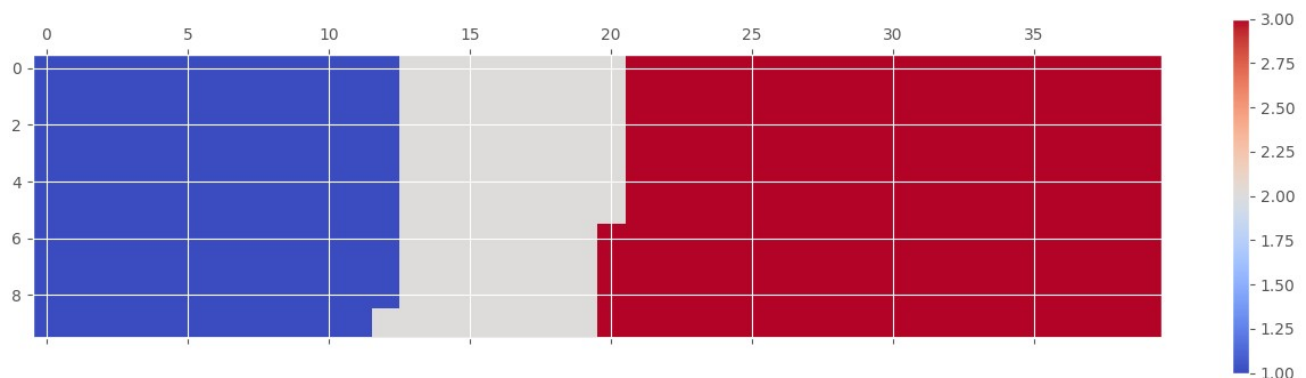
# define indices to loop through waffle chart
category_index = 0
tile_index = 0

# if the number of tiles populated for the current category is equal to its
# corresponding allocated tiles...
for col in range(width):
    for row in range(height):
        tile_index += 1
        if tile_index > sum(tiles_per_category[0:category_index]):
            # ...proceed to the next category
            category_index += 1

        # set the class value to an integer, which increases with class
        waffle_chart[row, col] = category_index
```

Step 5 - Visualize the matrix.

```
# use matshow to display the waffle chart
colormap = plt.cm.coolwarm
plt.matshow(waffle_chart, cmap=colormap)
plt.colorbar()
plt.show()
```



Step 6 - Add gridlines to separate tiles.

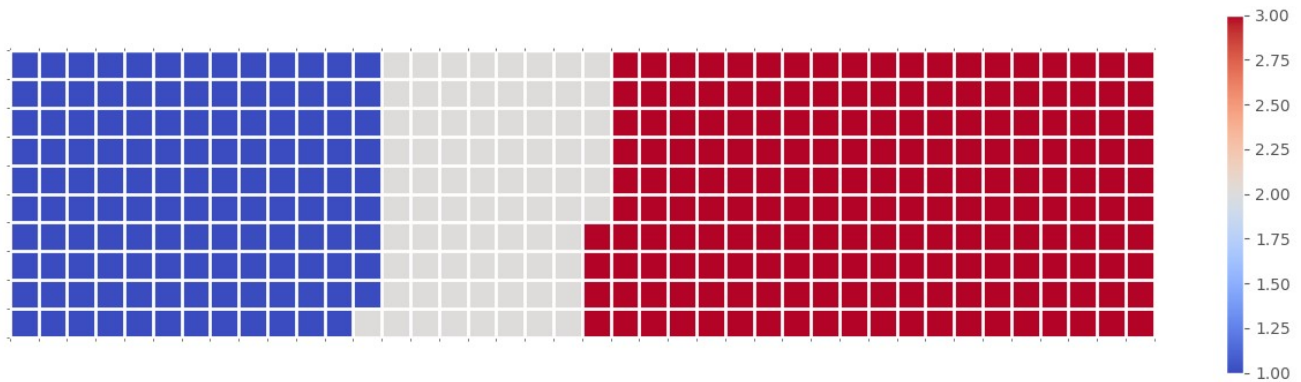
```
colormap = plt.cm.coolwarm
plt.matshow(waffle_chart, cmap=colormap)
plt.colorbar()

ax = plt.gca() # get the axis

# set minor ticks
ax.set_xticks(np.arange(-.5, width, 1), minor=True)
ax.set_yticks(np.arange(-.5, height, 1), minor=True)

# add gridlines based on minor ticks
ax.grid(which='minor', color='w', linestyle='-', linewidth=2)
```

```
plt.xticks([])
plt.yticks([])
plt.show()
```



Step 7 - Add a legend matching the color scheme.

```
colormap = plt.cm.coolwarm
plt.matshow(waffle_chart, cmap=colormap)
plt.colorbar()

ax = plt.gca() # get axis

# set minor ticks
ax.set_xticks(np.arange(-.5, width, 1), minor=True)
ax.set_yticks(np.arange(-.5, height, 1), minor=True)

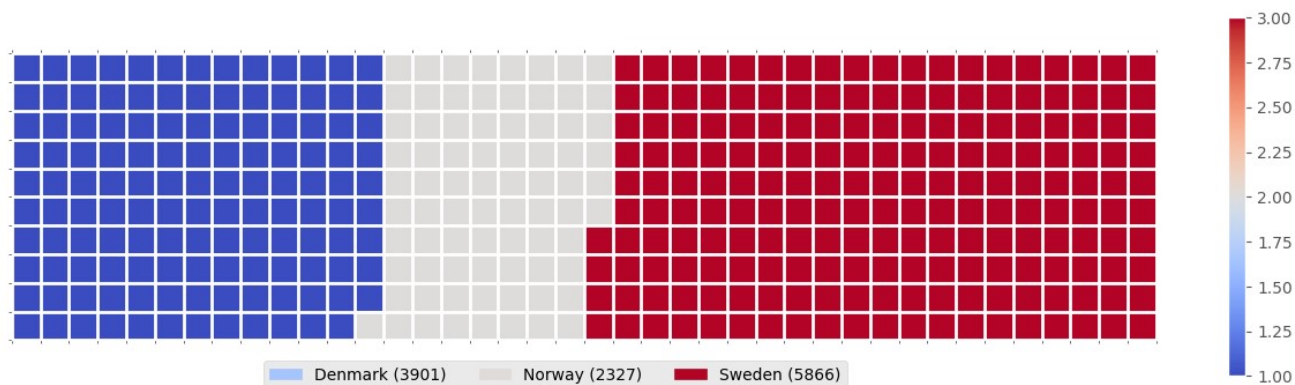
# add gridlines
ax.grid(which='minor', color='w', linestyle='-', linewidth=2)

plt.xticks([])
plt.yticks([])

# compute cumulative sum of individual categories to match color schemes between
chart and legend
values_cumsum = np.cumsum(df_dsn['Total'])
total_values = values_cumsum.iloc[-1]

# create legend
legend_handles = [
    mpatches.Patch(
        color=colormap(float(cumsum) / total_values),
        label=f"{cat} ({total})"
    )
    for cat, total, cumsum in zip(df_dsn.index, df_dsn['Total'], values_cumsum)]

# add Legend to chart
plt.legend(
    handles=legend_handles,
    loc='lower center',
    ncol=len(df_dsn.index.values),
    bbox_to_anchor=(0., -0.2, 0.95, .1))
plt.show()
```



Result : a polished waffle chart with a color bar and country legend.

Wrapping into a Reusable Function

Repeating all seven steps every time is inefficient, so I combine them into a single function called `create_waffle_chart`. This function would take the following parameters as input:

1. **categories:** Unique categories or classes in dataframe.
2. **values:** Values corresponding to categories or classes.
3. **height:** Defined height of waffle chart.
4. **width:** Defined width of waffle chart.
5. **colormap:** Colormap class
6. **value_sign:** more generalizable by adding this parameter to address signs that could be associated with a value such as %, \$, and so on. **value_sign** has a default value of empty string.

```
def create_waffle_chart(categories, values, height, width, colormap,
value_sign=''):
    total_values = sum(values)
    category_proportions = [float(v) / total_values for v in values]
    total_num_tiles = width * height
    print('Total number of tiles is', total_num_tiles)

    tiles_per_category = [round(p * total_num_tiles) for p in
category_proportions]
    for i, tiles in enumerate(tiles_per_category):
        print(categories[i] + ': ' + str(tiles))

    waffle_chart = np.zeros((height, width))
    category_index = 0
    tile_index = 0

    for col in range(width):
        for row in range(height):
            tile_index += 1
            if tile_index > sum(tiles_per_category[0:category_index]):
                category_index += 1
            waffle_chart[row, col] = category_index

    colormap = plt.cm.coolwarm
    plt.matshow(waffle_chart, cmap=colormap)
    plt.colorbar()
```

```

ax = plt.gca()
ax.set_xticks(np.arange(-.5, width, 1), minor=True)
ax.set_yticks(np.arange(-.5, height, 1), minor=True)
ax.grid(which='minor', color='w', linestyle='-', linewidth=2)
plt.xticks([])
plt.yticks([])

values_np = np.array(values)
values_cumsum = np.cumsum(values_np)
total_values = values_cumsum[-1]

legend_handles = []
for i, category in enumerate(categories):
    if value_sign == '%':
        label_str = f"{category} ({values[i]}{value_sign})"
    else:
        label_str = f"{category} ({value_sign}{values[i]})"
    color_val = colormap(float(values_cumsum[i] / total_values))
    legend_handles.append(mpatches.Patch(color=color_val, label=label_str))

plt.legend(
    handles=legend_handles,
    loc='lower center',
    ncol=len(categories),
    bbox_to_anchor=(0., -0.2, 0.95, .1))
plt.show()

```

Now I call it with the Denmark, Norway, Sweden data.

```

width = 40
height = 10
categories = df_dsn.index.values
values = df_dsn['Total']
colormap = plt.cm.coolwarm

```

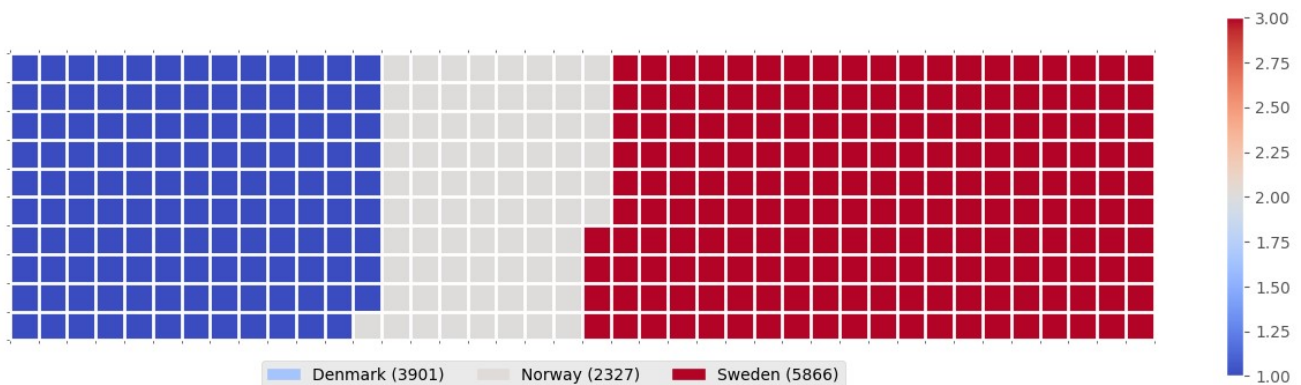
```
create_waffle_chart(categories, values, height, width, colormap)
```

Total number of tiles is 400

Denmark: 129

Norway: 77

Sweden: 194



Result: A fully labeled waffle chart showing the proportional immigrant contribution of Denmark,

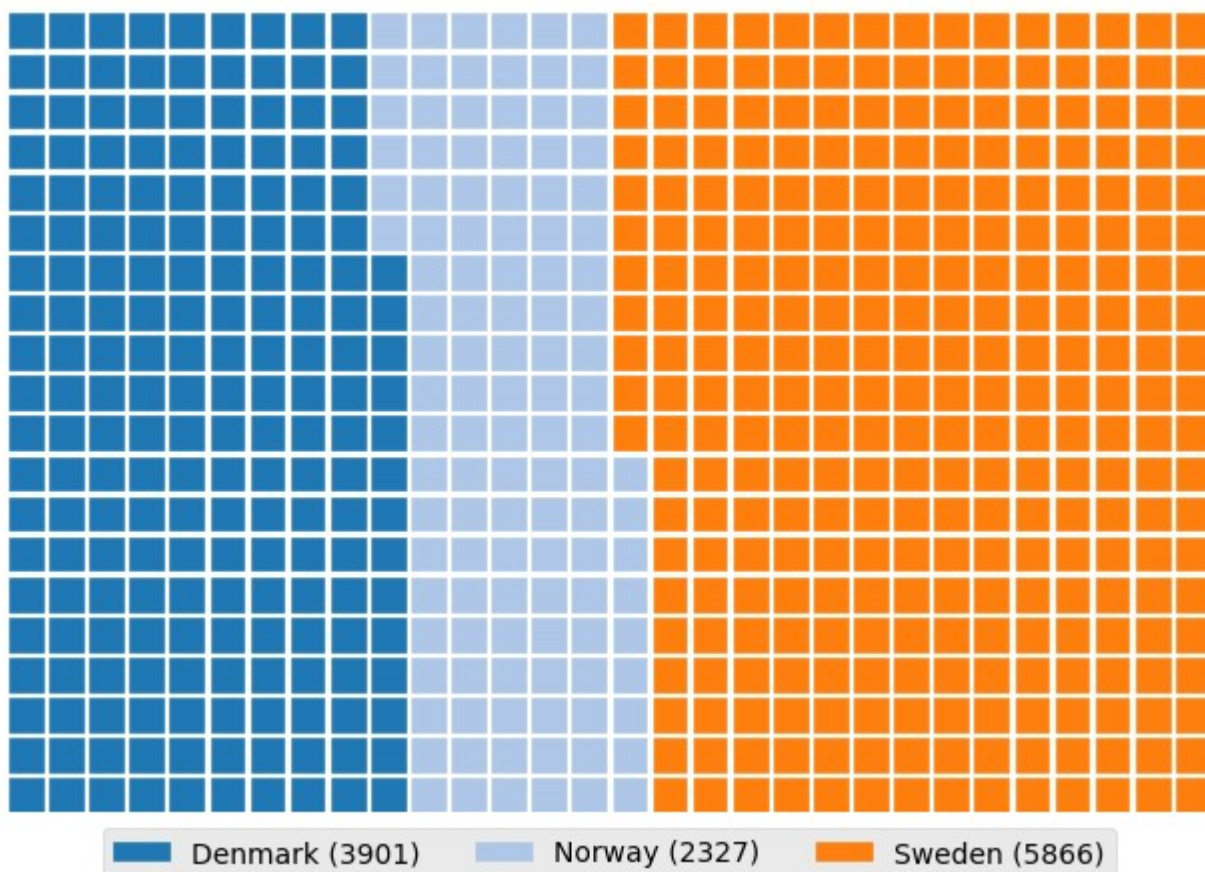
Norway, and Sweden.

Using PyWaffle (the Easier Way)

The `pywaffle` package provides a **built-in waffle chart class**.

```
from pywaffle import Waffle
```

```
fig = plt.figure(FigureClass=Waffle,  
                 rows=20, columns=30,  
                 values=df_dsn['Total'],  
                 cmap_name='tab20',  
                 legend = {'labels': [f"{k} ({v})" for k, v in  
zip(df_dsn.index.values,df_dsn.Total)],  
                           'loc': 'lower center',  
  
                           # fig bbox, (center, -ve y-axis)  
                           'bbox_to_anchor':(0.5,-0.1),  
                           'ncol': 3  
                           })  
plt.show()
```



Result: This produces the same waffle chart in a single cell.

Exercise: China and India Waffle Chart

Create a waffle chart showing the **proportional immigrant contribution** of China and India.

```
# Create dataframe for China and India
```

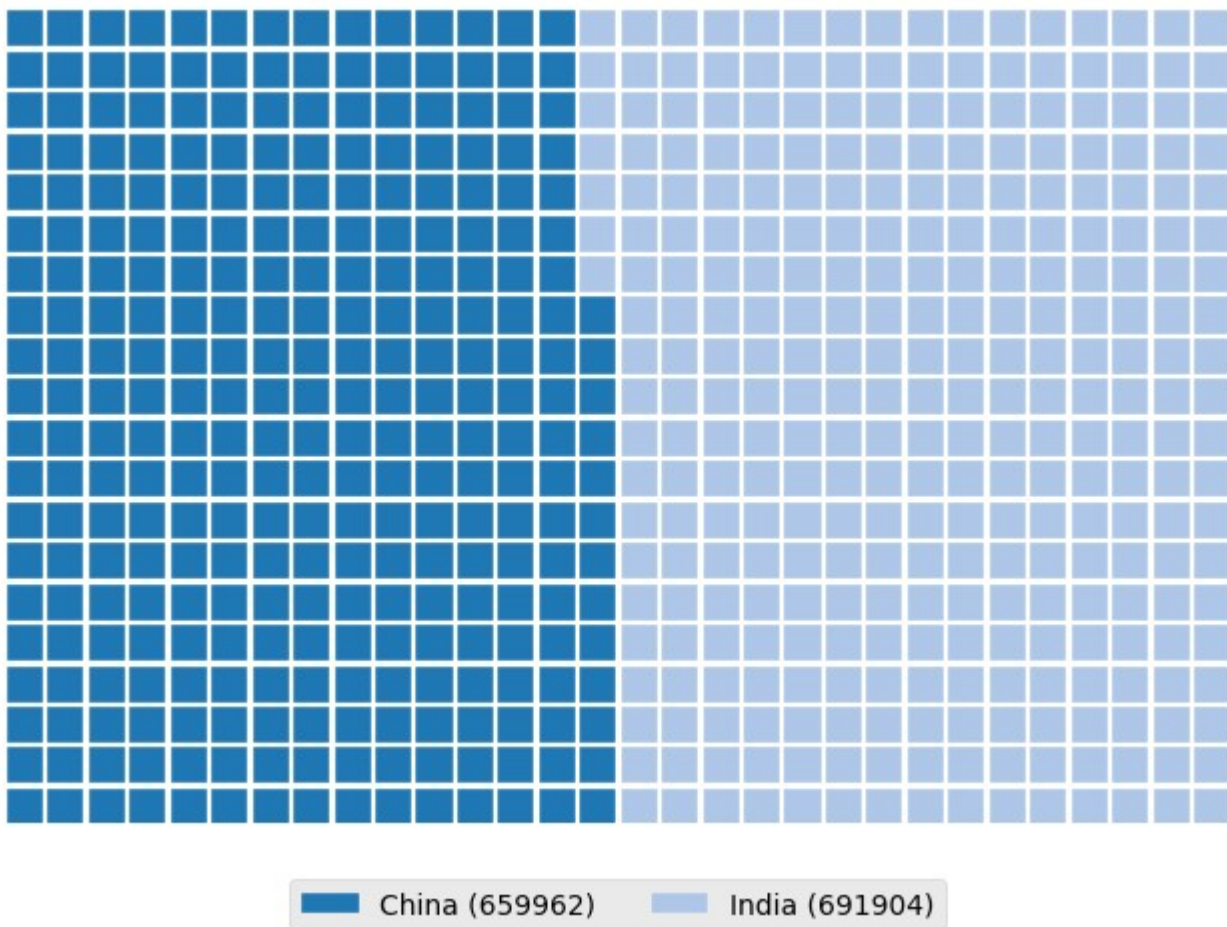
```

data_CI = df_can.loc[['China', 'India'], :]

# Set up the Waffle chart figure
fig = plt.figure(FigureClass=Waffle,
                 rows=20, columns=30,
                 values=data_CI['Total'],
                 cmap_name='tab20',
                 legend={
                     'labels': [f"{k} ({v})" for k, v in
zip(data_CI.index.values, data_CI.Total)],
                     'loc': 'center',
                     'bbox_to_anchor': (0.5, -0.1),
                     'ncol': 2
                 })

plt.show()

```



Result: A waffle chart visualizing that China and India together contributed a substantial portion of total Canadian immigration. The tile count directly reflects each country's proportional share.

4. Word Clouds

Word clouds ([link](#)) visualize **text frequency**: more frequent words appear **larger**. I start with short novel written by **Lewis Carroll** titled **Alice's Adventures in Wonderland**, then adapt the technique to the immigration dataset.

Word Cloud on Alice in Wonderland

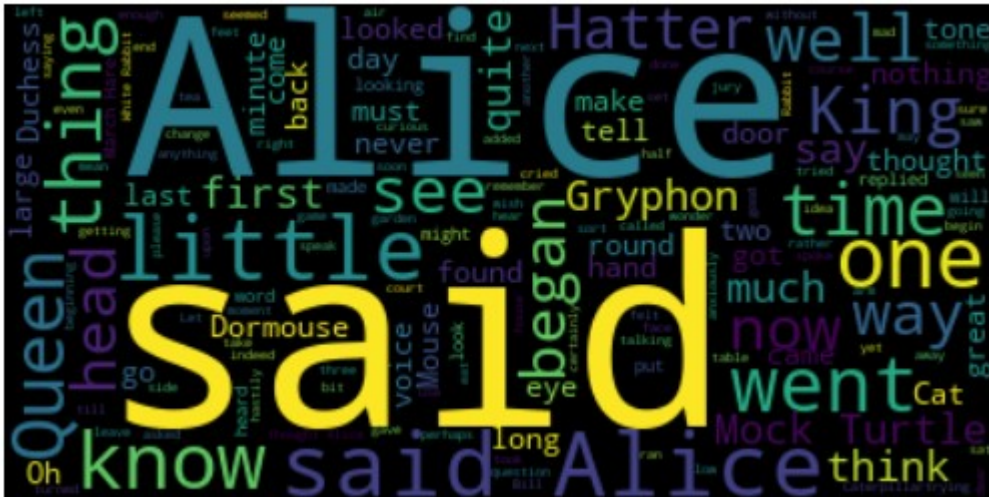
I load the novel text and create a basic word cloud.

```
alice_novel = urllib.request.urlopen(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/'
    'IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/'
    'alice_novel.txt'
).read().decode("utf-8")

stopwords = set(STOPWORDS)

alice_wc = WordCloud()
alice_wc.generate(alice_novel)

plt.imshow(alice_wc, interpolation='bilinear')
plt.axis('off')
plt.show()
```



Result: A word cloud where "Alice", "said", 'Queen' and "little" dominate - these are the most frequent words in the first 2000 tokens (Wordcloud **default**).

I remove the uninformative word "said" and regenerate.

```
stopwords = set(STOPWORDS)
stopwords.add('said')

alice_wc = WordCloud(stopwords=stopwords)
# prettier version
#alice_wc = WordCloud(stopwords=stopwords, background_color="white", width=800,
height=400)
alice_wc.generate(alice_novel)

fig = plt.figure(figsize=(14, 18))
plt.imshow(alice_wc, interpolation='bilinear')
plt.axis('off')
plt.show()
```



```

        # duplicate each country's name based on how much they contribute to the
        total immigration
        word_string += (country + ' ') * repeat_num_times

wordcloud = WordCloud(background_color='black').generate(word_string)

plt.figure(figsize=(14, 18))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()

```



Result: The largest country names in the cloud - such as India, China, Philippines - correspond to the top sources of Canadian immigrants. The top ~15 single-word countries are most prominent.

Now I superimpose the word cloud on top of the map of Canada.

```

# Load Canada map
canada_img = Image.open(r"D:\
jupy_master_base_env\jupy_projects\IBM_Data_C008_T001_L01\canada_map.jpg").convert(
# convert to grayscale

# Convert to numpy array
canada_array = np.array(canada_img)

# Threshold: make Canada shape white (255), background black (0)
canada_mask = np.where(canada_array > 128, 255, 0).astype(np.uint8)

wordcloud = WordCloud(
    background_color="white",
    max_words=90,
    mask=canada_mask,
).generate(word_string)

plt.figure(figsize=(14, 18))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")

```

```
# Overlay the mask (white silhouette of Canada)
plt.imshow(canada_mask, cmap="gray", alpha=0.3) # alpha (0.3-1.0) for visibility
plt.show()
```



```
#if you get attribute error while generating worldcloud, upgrade Pillow and numpy
using below code
#%pip install --upgrade Pillow
#%pip install --upgrade numpy
```

5. Plotting with Seaborn

Seaborn makes **statistical visualizations concise and attractive**. More **efficient compared to scatter** plot. I use it for categorical plots and regression plots against the immigration data.

Categorical Plots - **Countplot**

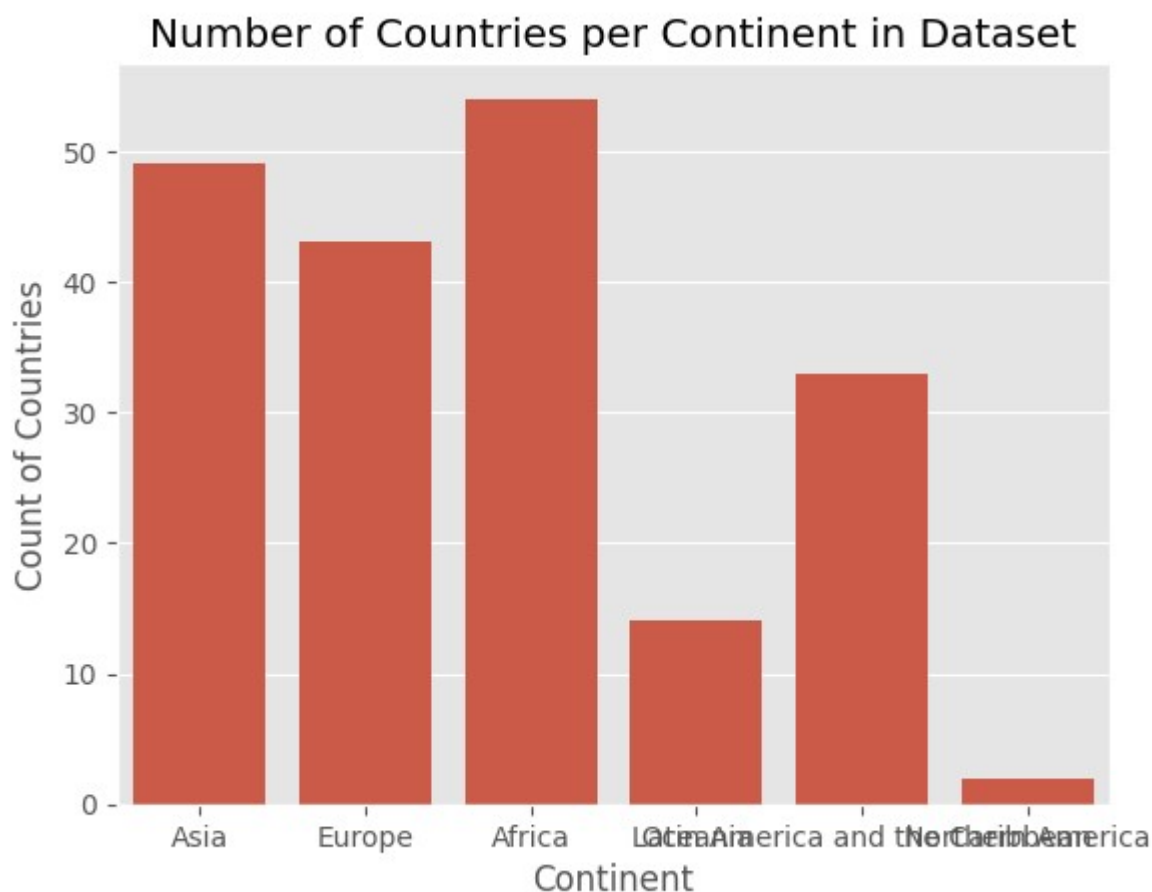
First, I inspect which continents appear in the data.

```
df_can['Continent'].unique()
```

```
<StringArray>  
[  
    'Asia',  
    'Africa',  
    'Latin America and the Caribbean',  
    'Europe',  
    'Oceania',  
    'Northern America']  
Length: 6, dtype: str
```

A **countplot** shows the frequency of each continent (for **categorical** variable).

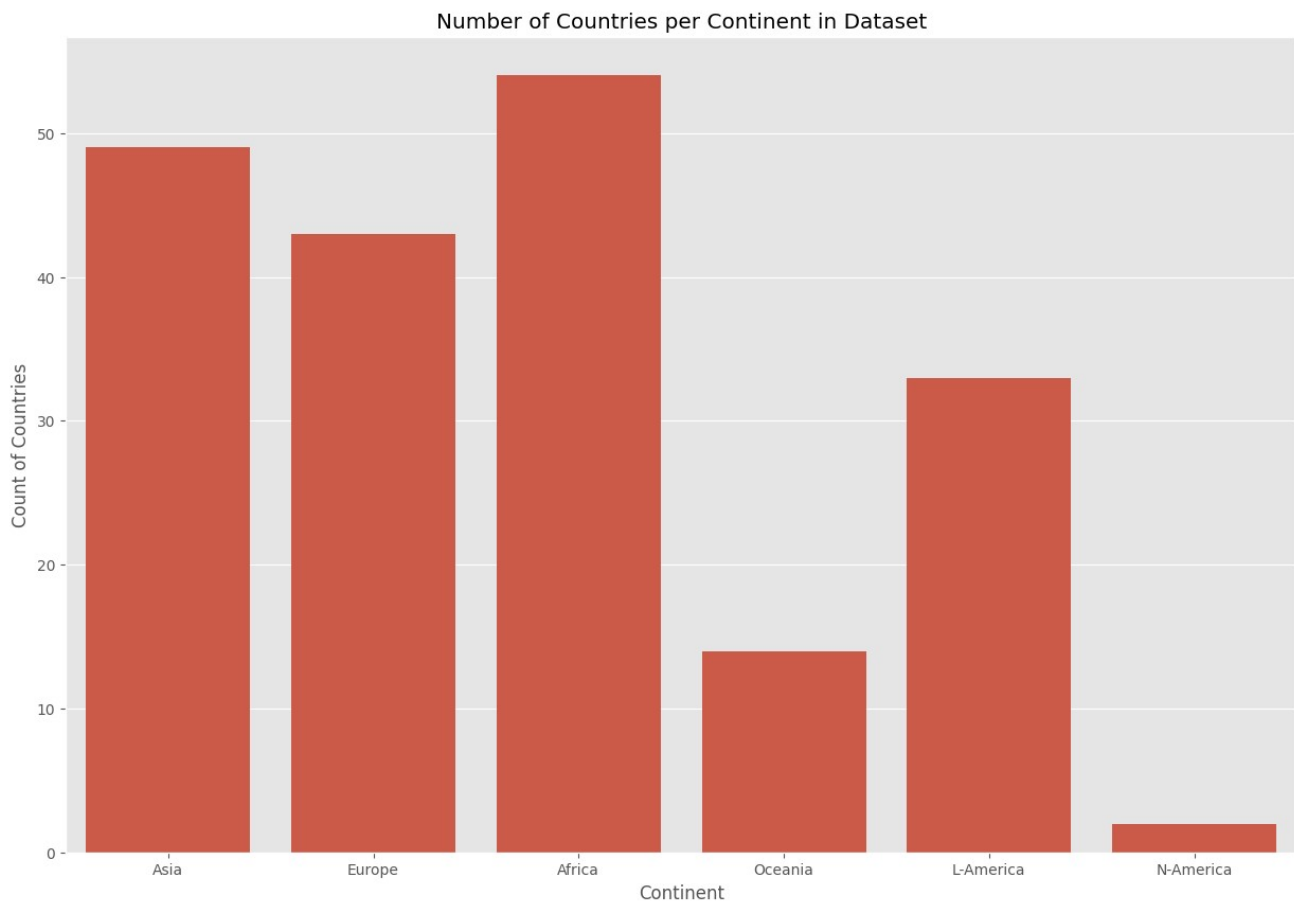
```
sns.countplot(x='Continent', data=df_can)  
plt.title("Number of Countries per Continent in Dataset")  
plt.ylabel("Count of Countries");
```



Result: The x-axis labels are cramped because "Latin America and the Caribbean" is too long. I shorten the names and replot.

```
df_can1 = df_can.replace('Latin America and the Caribbean', 'L-America')  
df_can1 = df_can1.replace('Northern America', 'N-America')
```

```
plt.figure(figsize=(15, 10))  
sns.countplot(x='Continent', data=df_can1)  
plt.title("Number of Countries per Continent in Dataset")  
plt.ylabel("Count of Countries");
```

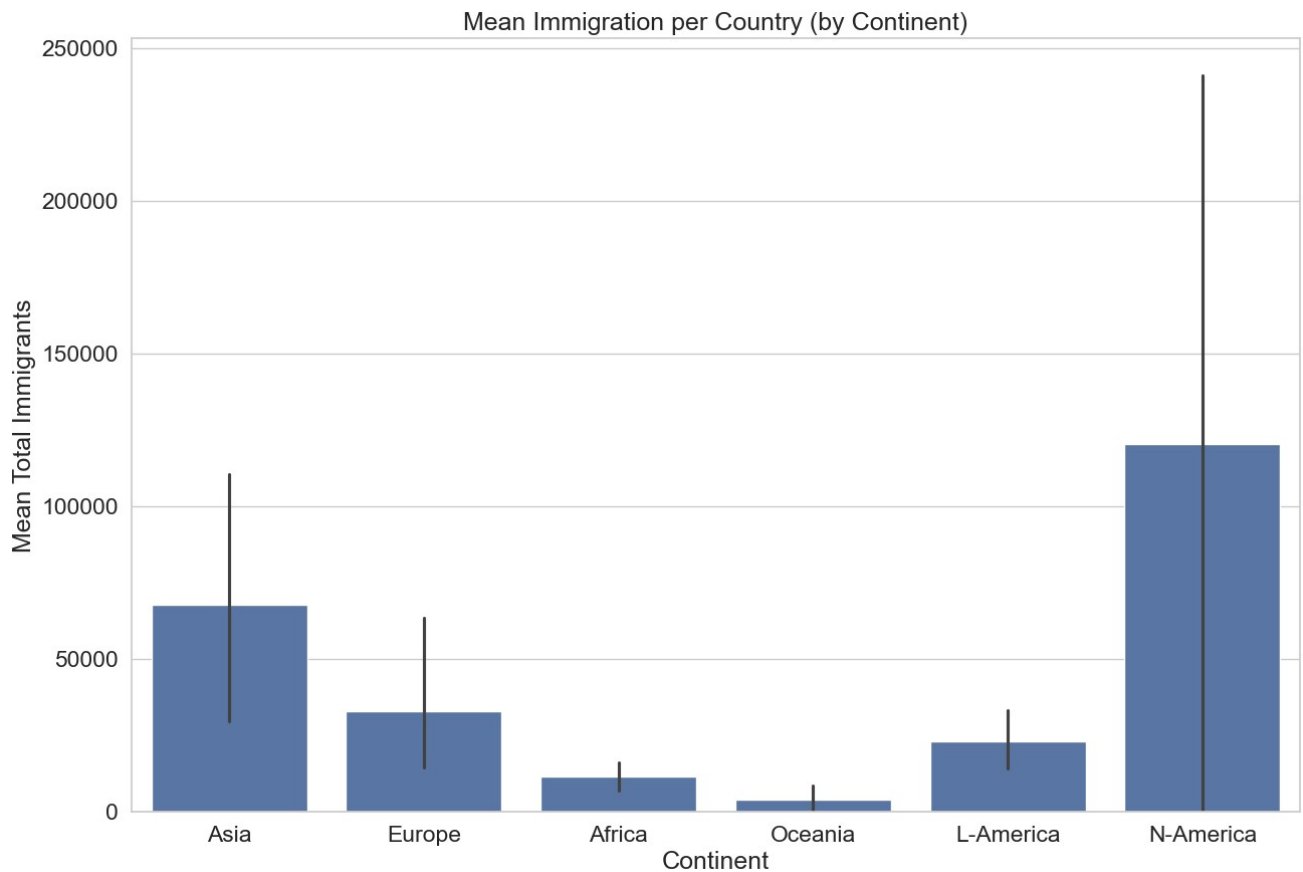


Result: A clean bar chart **showing how many countries per continent are in the dataset**. Europe and Asia lead, with the fewest in Northern America (**only USA and Canada**) and Oceania.

Categorical Plots - **Barplot**

A barplot **aggregates a numeric column** (Total) **by a categorical column** (Continent), showing mean values with confidence intervals.

```
plt.figure(figsize=(15, 10))
sns.barplot(x='Continent', y='Total', data=df_can1)
plt.title("Mean Immigration per Country (by Continent)")
plt.ylabel("Mean Total Immigrants");
```



Result: The **mean total immigration per continent**. Europe and Asia have high averages; Africa and Oceania have lower ones. N-America looks highest because only **USA + Canada**.

I verify this with a manual groupby:

```
df_can1.groupby('Continent')['Total'].mean()
```

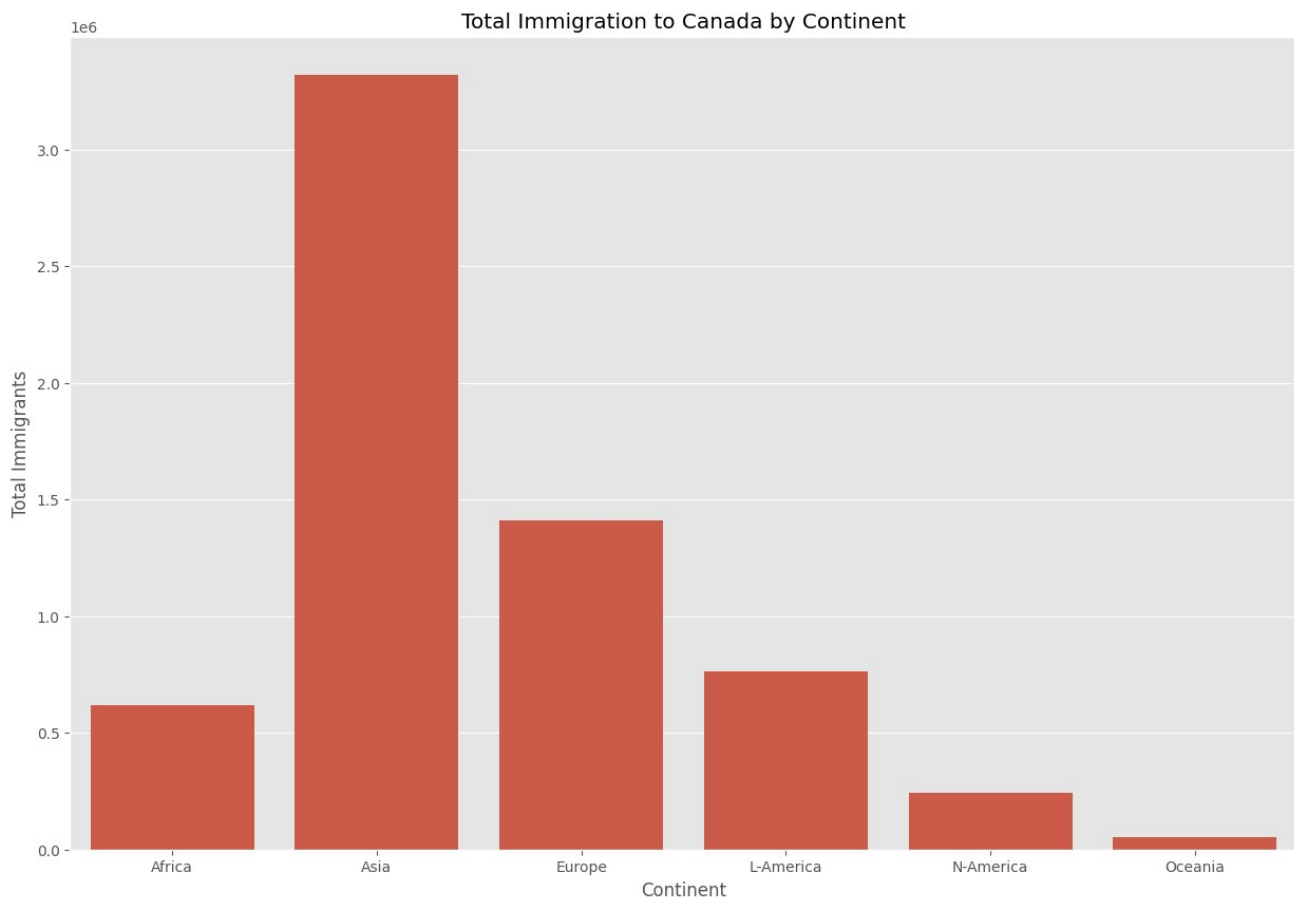
```
Continent
Africa      11462.000000
Asia        67710.081633
Europe       32812.720930
L-America   23186.303030
N-America   120571.000000
Oceania       3941.000000
Name: Total, dtype: float64
```

Now I plot the total immigration per continent

```
# Group by continent and sum the totals
df_continent_totals = df_can1.groupby('Continent', as_index=False)['Total'].sum()

# Plot the summed totals
plt.figure(figsize=(15, 10))
sns.barplot(x='Continent', y='Total', data=df_continent_totals, errorbar=None)

plt.title("Total Immigration to Canada by Continent")
plt.ylabel("Total Immigrants");
```



Result: Bar plot showing **total immigration by continent**.

6. Regression Plots

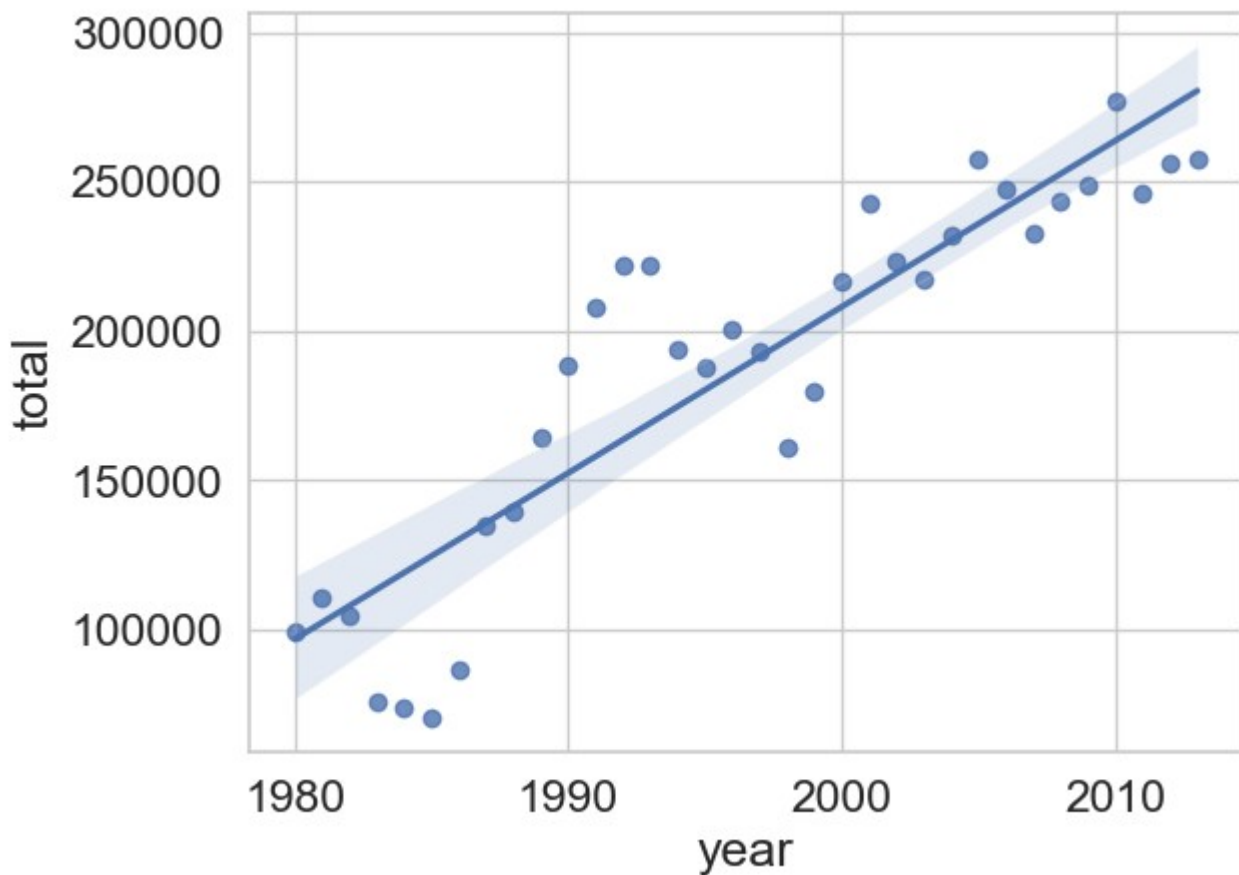
I now create a **regression plot** for total Canadian immigration across all countries, year by year. First, I build the aggregated dataframe.

```
years = list(map(str, range(1980, 2014)))
# total population per year
df_tot = pd.DataFrame(df_can[years].sum(axis=0))
# set as float for regression
df_tot.index = map(float, df_tot.index)
# reset the index to put in back in as a column in the df_tot dataframe
df_tot.reset_index(inplace=True)
df_tot.columns = ['year', 'total'] # rename
df_tot.head()
```

	year	total
0	1980.0	99137
1	1981.0	110563
2	1982.0	104271
3	1983.0	75550
4	1984.0	73417

Now the regression plot in its most basic form:

```
sns.regplot(x='year', y='total', data=df_tot);
```

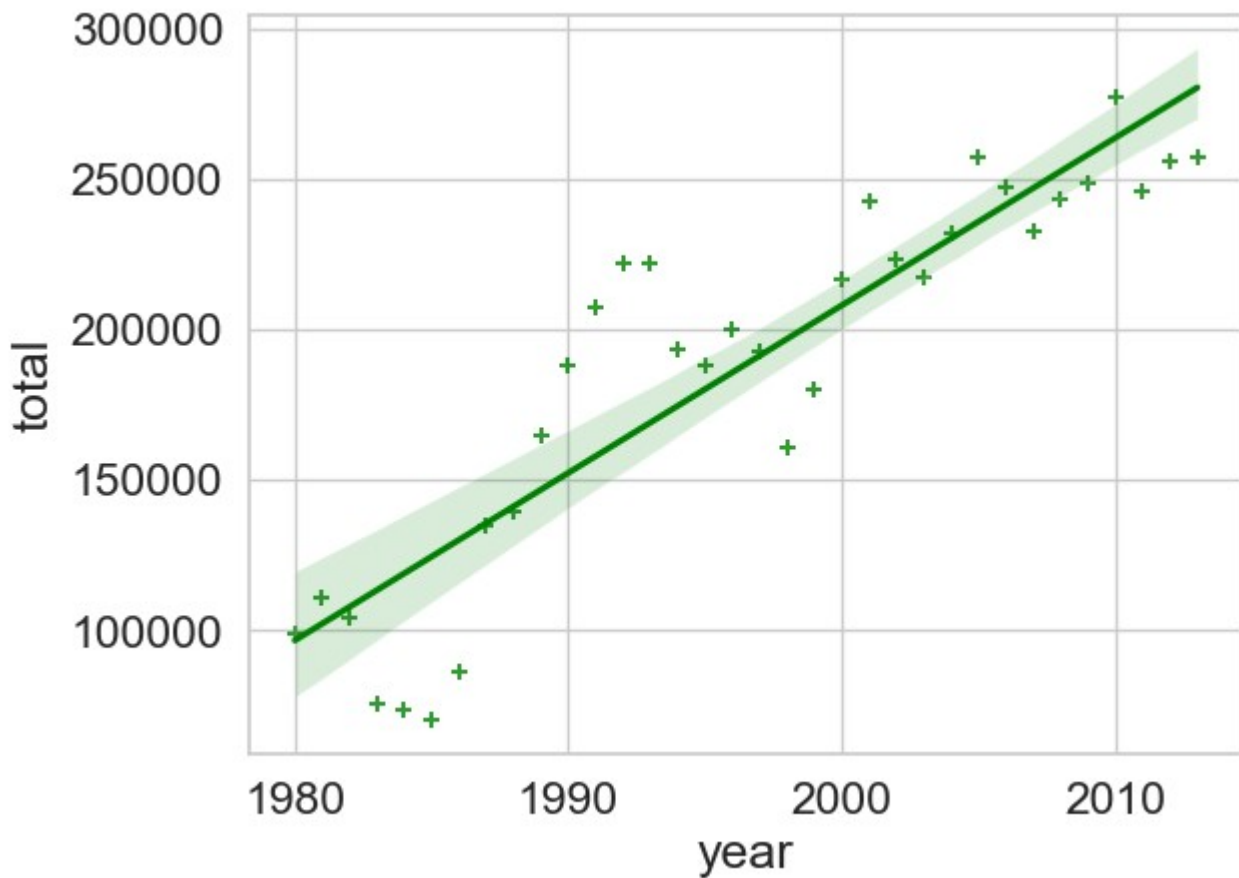


Result: A scatter plot with a blue regression line and a **shaded confidence interval** - showing a clear upward trend in total immigration.

I customize the appearance step by step.

Green markers with + shape:

```
sns.regplot(x='year', y='total', data=df_tot, color='green', marker='+')  
plt.show()
```



Larger figure and larger markers:

```
plt.figure(figsize=(15, 10))
ax = sns.regplot(x='year', y='total', data=df_tot, color='green',
                 marker='+', scatter_kws={'s': 200})
ax.set(xlabel='Year', ylabel='Total Immigration')
ax.set_title('Total Immigration to Canada from 1980 - 2013')
plt.show()
```

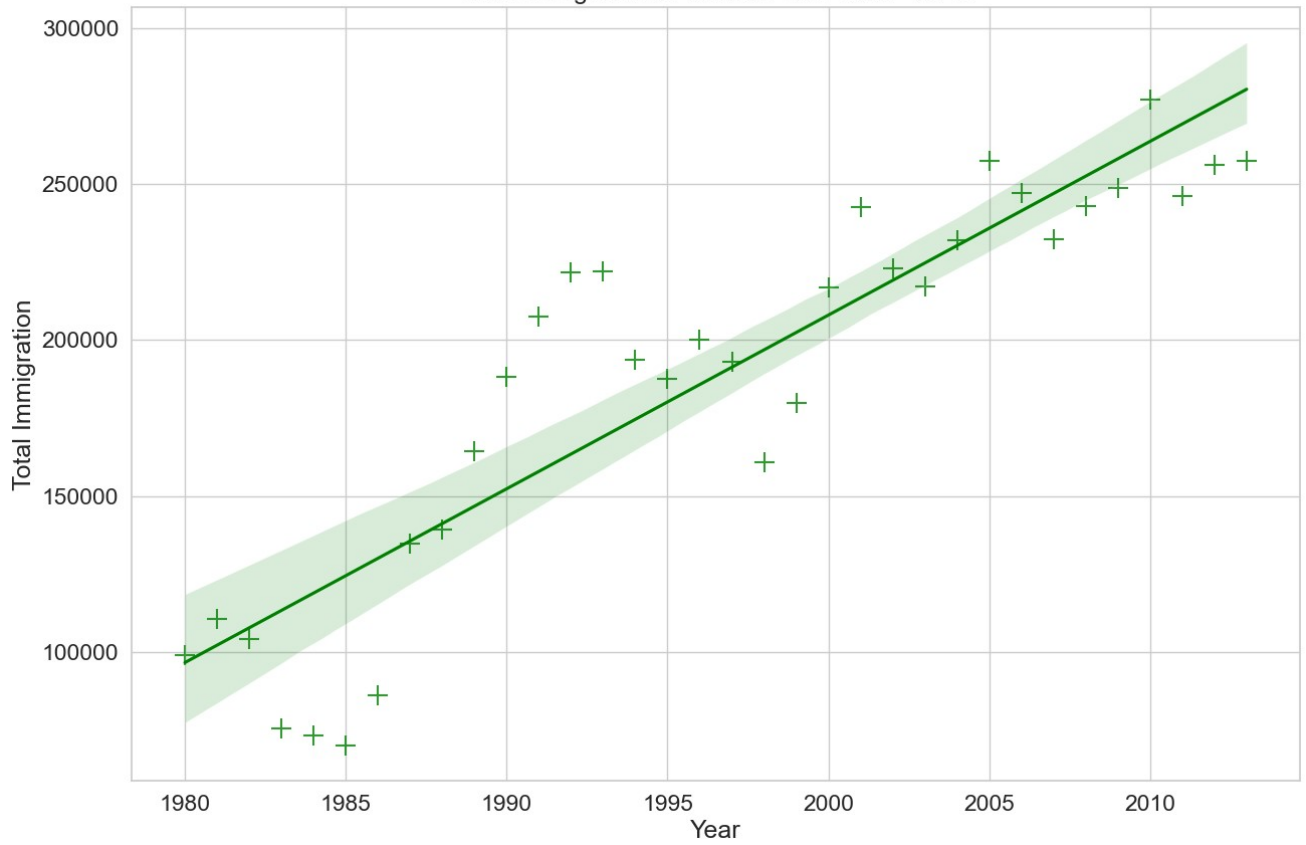


White background with gridlines and larger fonts:

```
plt.figure(figsize=(15, 10))
sns.set(font_scale=1.5)
sns.set_style('whitegrid') # change background to white background
#sns.set_style('ticks') # white plain bg

ax = sns.regplot(x='year', y='total', data=df_tot, color='green',
                 marker='+', scatter_kws={'s': 200})
ax.set(xlabel='Year', ylabel='Total Immigration')
ax.set_title('Total Immigration to Canada from 1980 - 2013')
plt.show()
```

Total Immigration to Canada from 1980 - 2013



Result: A clean, publication-ready regression plot showing that total immigration to Canada grew steadily from ~100,000 in 1980 to over 250,000 by 2013 - a clear positive linear trend captured in just a few lines of Seaborn code.

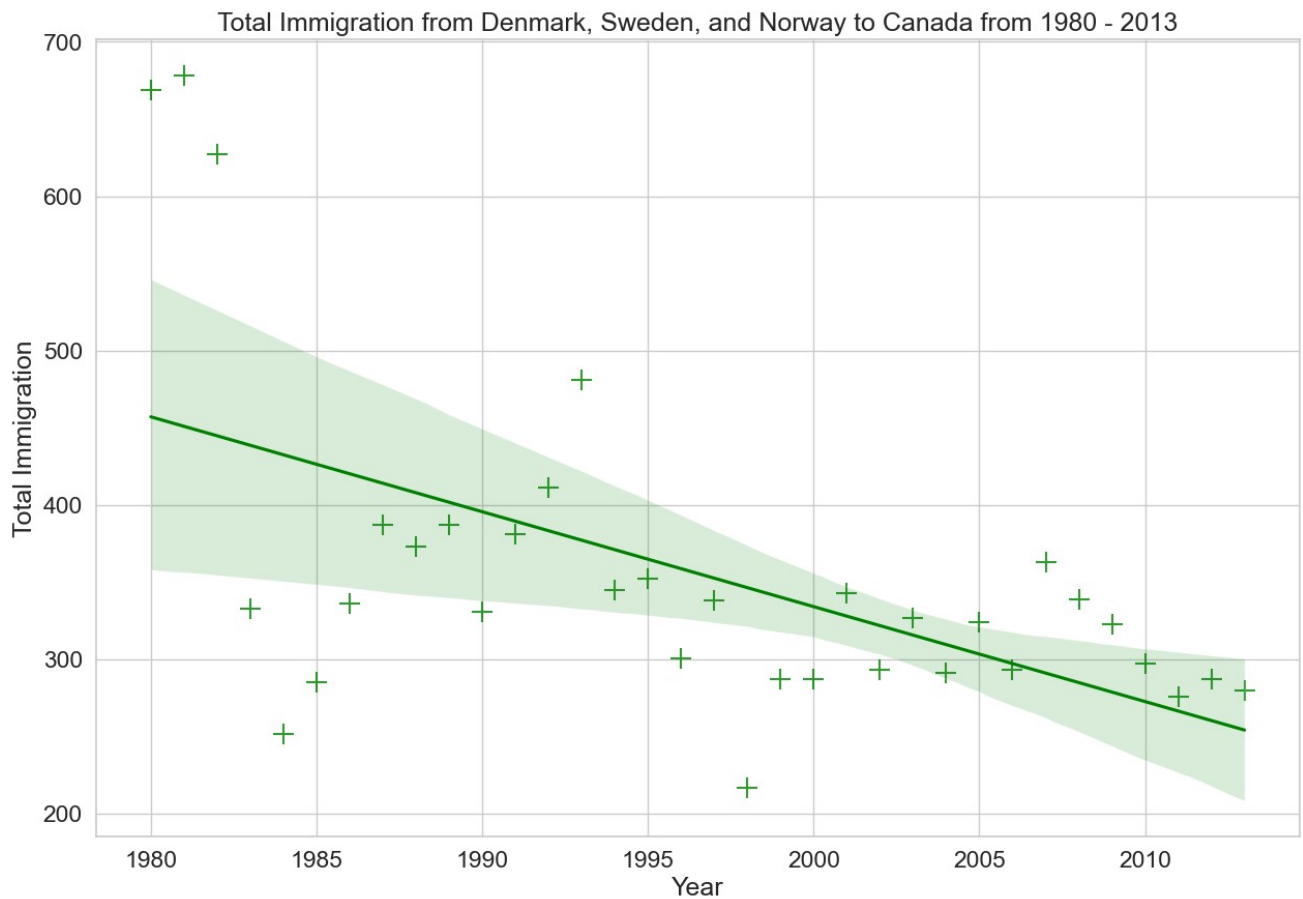
Exercise: Regression Plot for Denmark, Norway, Sweden

Use Seaborn to create a scatter plot with a regression line showing total immigration from Denmark, Sweden, and Norway from 1980 to 2013.

```
years = list(map(str, range(1980, 2014)))
# create df_countries dataframe
df_countries = df_can.loc[['Denmark', 'Norway', 'Sweden'], years].transpose()
# summing across three countries for each year
df_total = pd.DataFrame(df_countries.sum(axis=1))
df_total.reset_index(inplace=True)
df_total.columns = ['year', 'total']
# change column year from string to int to create scatter plot
df_total['year'] = df_total['year'].astype(int)

plt.figure(figsize=(15, 10))
sns.set(font_scale=1.5)
sns.set_style('whitegrid')

ax = sns.regplot(x='year', y='total', data=df_total,
                 color='green', marker='+', scatter_kws={'s': 200})
ax.set(xlabel='Year', ylabel='Total Immigration')
ax.set_title('Total Immigration from Denmark, Sweden, and Norway to Canada from
1980 - 2013')
plt.show()
```



Result: The regression line reveals a slight downward trend - immigration from these three Scandinavian countries to Canada has decreased modestly over the 34-year period, though year-to-year variation is notable.

That's the full walkthrough of creating waffle charts, word clouds, and regression plots - three distinct visualization techniques applied to the Canadian immigration dataset to reveal proportional distributions, textual patterns, and multi-decade trends.